

Project code.py – exported 2025-08-16 01:43

```
1 | # ===== Export terminal output to pdf =====
2 | import os, random, gc
3 | import sys, io, atexit
4 | class Tee:
5 |     def __init__(self, *files): self.files = files
6 |     def write(self, x):
7 |         for f in self.files: f.write(x)
8 |     def flush(self):
9 |         for f in self.files: f.flush()
10 |
11 | _log = open("run_output.txt", "w", encoding="utf-8")
12 | sys.stdout = Tee(sys.stdout, _log) # print() goes to console + file
13 | sys.stderr = Tee(sys.stderr, _log) # errors too
14 |
15 | @atexit.register
16 | def _close_log():
17 |     try: _log.close()
18 |     except: pass
19 |
20 |
21 | # ===== DETERMINISM HEADER =====
22 | import os, random, gc
23 |
24 | SEED = 445 # single source of truth
25 | DETERMINISTIC = True # flip to False for speed over strict repeatability
26 | THREADS = 1 if DETERMINISTIC else max(1, os.cpu_count() // 2)
27 |
28 | # Make numeric libs single-threaded to avoid nondeterministic reductions
29 | os.environ["OMP_NUM_THREADS"] = "1" if DETERMINISTIC else str(THREADS)
30 | os.environ["OPENBLAS_NUM_THREADS"] = "1" if DETERMINISTIC else str(THREADS)
31 | os.environ["MKL_NUM_THREADS"] = "1" if DETERMINISTIC else str(THREADS)
32 | os.environ["VECLIB_MAXIMUM_THREADS"] = "1" if DETERMINISTIC else str(THREADS)
33 | os.environ["NUMEXPR_NUM_THREADS"] = "1" if DETERMINISTIC else str(THREADS)
34 |
35 | # Python & NumPy RNG
36 | os.environ["PYTHONHASHSEED"] = str(SEED)
37 | random.seed(SEED)
38 | # =====
39 |
40 | # Core libs
41 | import pandas as pd
42 | import numpy as np
43 | from pathlib import Path
44 |
```

Project code.py – exported 2025-08-16 01:43

```
45 | # Sklearn base imports
46 | from sklearn.impute import SimpleImputer
47 | from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV
48 | from sklearn.preprocessing import StandardScaler
49 | from sklearn.pipeline import Pipeline
50 | from sklearn.linear_model import LogisticRegression
51 | from sklearn.metrics import (
52 |     roc_auc_score, roc_curve, precision_recall_curve, average_precision_score,
53 |     confusion_matrix, classification_report
54 | )
55 |
56 | # Model libraries
57 | from xgboost import XGBClassifier
58 | from catboost import CatBoostClassifier
59 | from sklearn.ensemble import RandomForestClassifier
60 |
61 | # Plotting
62 | import matplotlib.pyplot as plt
63 |
64 | # For reproducibility of random numbers in simulations
65 | import hashlib
66 |
67 | # ===== Load Data & Data Pre-processing =====
68 | # --- file paths (edit TEST_PATH if needed) ---
69 | # r"H:\git\ML-Credit-Risk\Dataset\cs-training.csv"
70 | # r"D:\Github\ML-Credit-Risk\Dataset\GiveMeSomeCredit\cs-training.csv"
71 |
72 | df = pd.read_csv(r"H:\git\ML-Credit-Risk\Dataset\cs-training.csv")
73 |
74 | # drop index column if exists
75 | df = df.drop(columns=["Unnamed: 0"], errors="ignore")
76 |
77 | # keep ages 20–100
78 | df = df[(df["age"] >= 20) & (df["age"] <= 100)].copy()
79 |
80 | # MonthlyIncome: keep 0s, impute only NAs by age-group median ===
81 | df["NoIncomeFlag"] = (df["MonthlyIncome"] == 0).astype("int8")
82 |
83 | bins = [20, 30, 40, 50, 60, 70, 80, 90, 100]
84 | labels = ["20-29", "30-39", "40-49", "50-59", "60-69", "70-79", "80-89", "90-100"]
85 | df["age_group"] = pd.cut(df["age"], bins=bins, labels=labels, right=False)
86 |
87 | median_by_group = (
88 |     df.loc[df["MonthlyIncome"] > 0]
```

Project code.py – exported 2025-08-16 01:43

```
89 |         .groupby("age_group", observed=False)["MonthlyIncome"]
90 |         .median()
91 |     )
92 |     mi_na_mask = df["MonthlyIncome"].isna()
93 |     df.loc[mi_na_mask, "MonthlyIncome"] = df.loc[mi_na_mask, "age_group"].map(median_by_group)
94 |     df["IncomeImputedFlag"] = mi_na_mask.astype("int8")
95 |     df = df.drop(columns=["age_group"])
96 |
97 |     # DebtRatio: coerce, remove inf, and cap per new rules (0-income=10, imputed=2, normal=10) ===
98 |     dr = pd.to_numeric(df["DebtRatio"], errors="coerce")
99 |     dr = dr.mask(np.isinf(dr), np.nan) # remove ±inf safely
100 |
101 |     # Masks from existing columns/flags
102 |     pos_income = df["MonthlyIncome"] > 0
103 |     was_imputed = df["IncomeImputedFlag"].eq(1) # MonthlyIncome was NA and then imputed by median
104 |     zero_income = df["NoIncomeFlag"].eq(1) # MonthlyIncome == 0 (flag set before imputation)
105 |     normal_income = pos_income & (~was_imputed) # positive and not imputed
106 |
107 |     # Caps
108 |     REALISTIC_CAP = 10.0
109 |     IMPUTED_CAP = 2.0
110 |
111 |     # Start from a clean series
112 |     dr_series = pd.Series(dr, index=df.index)
113 |
114 |     # Apply caps with lower bound at 0 (avoid negatives)
115 |     dr_series.loc[was_imputed] = np.clip(dr_series.loc[was_imputed], 0.0, IMPUTED_CAP)
116 |     dr_series.loc[zero_income] = np.clip(dr_series.loc[zero_income], 0.0, REALISTIC_CAP)
117 |     dr_series.loc[normal_income] = np.clip(dr_series.loc[normal_income], 0.0, REALISTIC_CAP)
118 |
119 |     # Fill any remaining NaNs with the median of observed values
120 |     dr_series = dr_series.fillna(np.nanmedian(dr_series.values))
121 |     df["DebtRatio"] = dr_series.astype(float)
122 |
123 |     # NumberOfDependents: flag missing, top-code, impute median, large-family flag
124 |     dep_na_mask = df["NumberOfDependents"].isna()
125 |     df["DependentsMissingFlag"] = dep_na_mask.astype("int8")
126 |
127 |     # top-code extremes at 5 before imputation
128 |     dep_series = df["NumberOfDependents"].copy()
129 |     dep_series = dep_series.where(dep_series.isna() | (dep_series <= 5), 5)
130 |     dep_median = int(dep_series.median(skipna=True))
131 |     dep_series = dep_series.fillna(dep_median).astype("int8")
132 |
```

Project code.py – exported 2025-08-16 01:43

```
133 | df["NumberOfDependents"] = dep_series
134 | df["LargeFamilyFlag"] = (df["NumberOfDependents"] >= 5).astype("int8")
135 |
136 | # Number of Times Past Due Clean up
137 | cols_past_due = [
138 |     "NumberOfTime30-59DaysPastDueNotWorse",
139 |     "NumberOfTime60-89DaysPastDueNotWorse",
140 |     "NumberOfTimes90DaysLate"
141 | ]
142 | for col in cols_past_due:
143 |     df[col] = pd.to_numeric(df[col], errors="coerce")
144 |     df[f"{col}_MissingFlag"] = df[col].isin([96, 98]).astype(int)
145 |     df[col] = df[col].replace({96: np.nan, 98: np.nan})
146 |     median_val = df[col].median(skipna=True)
147 |     df[col] = df[col].fillna(median_val)
148 |     df[col] = np.where(df[col] > 10, 10, df[col]).astype(int)
149 |
150 | # ===== Split the dataset =====
151 | TARGET = "SeriousDlqin2yrs"
152 | X = df.drop(columns=[TARGET])
153 | y = df[TARGET].astype(int)
154 |
155 | # First split into train+val and test (stratified)
156 | X_trainval, X_test, y_trainval, y_test = train_test_split(
157 |     X, y, test_size=0.2, stratify=y, random_state=SEED
158 | )
159 | X_train, X_val, y_train, y_val = train_test_split(
160 |     X_trainval, y_trainval, test_size=0.25, stratify=y_trainval, random_state=SEED
161 | )
162 |
163 | # ----- Persist splits so future runs use identical rows -----
164 | split_dir = Path("./_splits"); split_dir.mkdir(exist_ok=True)
165 | f_train = split_dir / "train_idx.npy"
166 | f_val = split_dir / "val_idx.npy"
167 | f_test = split_dir / "test_idx.npy"
168 |
169 | if all(f.exists() for f in (f_train, f_val, f_test)):
170 |     # Rebuild splits from saved indices
171 |     idx_train = np.load(f_train, allow_pickle=False)
172 |     idx_val = np.load(f_val, allow_pickle=False)
173 |     idx_test = np.load(f_test, allow_pickle=False)
174 |     X_train, y_train = X.loc[idx_train], y.loc[idx_train]
175 |     X_val, y_val = X.loc[idx_val], y.loc[idx_val]
176 |     X_test, y_test = X.loc[idx_test], y.loc[idx_test]
```

Project code.py – exported 2025-08-16 01:43

```
177 | else:
178 |     # First run: save the indices we just generated with the fixed random_state
179 |     np.save(f_train, X_train.index.values)
180 |     np.save(f_val, X_val.index.values)
181 |     np.save(f_test, X_test.index.values)
182 |
183 |
184 | # ===== L1 Logistic Regression =====
185 | pipe = Pipeline(steps=[
186 |     ("scaler", StandardScaler()),
187 |     ("clf", LogisticRegression(
188 |         penalty="l1",
189 |         solver="saga",
190 |         max_iter=5000,
191 |         class_weight="balanced",
192 |         random_state=SEED,
193 |         n_jobs=THREADS
194 |     ))
195 | ])
196 |
197 | cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=SEED)
198 |
199 | param_grid = {"clf__C": np.logspace(-3, 1, 16)}
200 |
201 | grid = GridSearchCV(
202 |     estimator=pipe,
203 |     param_grid=param_grid,
204 |     scoring="roc_auc",
205 |     cv=cv,
206 |     n_jobs=1,
207 |     verbose=1
208 | )
209 |
210 | grid.fit(X_train, y_train)
211 | best_model = grid.best_estimator_
212 | print("Best C:", grid.best_params_["clf__C"])
213 | print("CV ROC-AUC (mean):", grid.best_score_)
214 |
215 | # Validate on VAL set
216 | val_proba = best_model.predict_proba(X_val)[: , 1]
217 | val_auc = roc_auc_score(y_val, val_proba)
218 | val_prauc = average_precision_score(y_val, val_proba)
219 |
220 | # KS statistic
```

Project code.py – exported 2025-08-16 01:43

```
221 | fpr, tpr, thr = roc_curve(y_val, val_proba)
222 | ks_vals = tpr - fpr
223 | ks = np.max(ks_vals)
224 | thr_ks = thr[np.argmax(ks_vals)]
225 |
226 | print(f"Validation ROC-AUC: {val_auc:.4f}")
227 | print(f"Validation PR-AUC : {val_prauc:.4f}")
228 | print(f"Validation KS      : {ks:.4f} @ threshold {thr_ks:.4f}")
229 |
230 | # Choose threshold by max KS on validation
231 | val_pred = (val_proba >= thr_ks).astype(int)
232 | print("\nValidation Confusion Matrix:")
233 | print(confusion_matrix(y_val, val_pred))
234 | print("\nValidation Classification Report:")
235 | print(classification_report(y_val, val_pred, digits=4))
236 |
237 | # Retrain on TRAIN+VAL with best hyperparams, then evaluate on TEST
238 | best_model_final = GridSearchCV(
239 |     estimator=pipe,
240 |     param_grid={"clf__C": [grid.best_params_["clf__C"]]},
241 |     scoring="roc_auc",
242 |     cv=cv,
243 |     n_jobs=-1
244 | ).fit(pd.concat([X_train, X_val]), pd.concat([y_train, y_val])).best_estimator_
245 |
246 | test_proba = best_model_final.predict_proba(X_test)[: , 1]
247 | test_auc   = roc_auc_score(y_test, test_proba)
248 | test_prauc = average_precision_score(y_test, test_proba)
249 | fpr_t, tpr_t, thr_t = roc_curve(y_test, test_proba)
250 | ks_t = np.max(tpr_t - fpr_t)
251 |
252 | # Use the threshold chosen on validation
253 | test_pred = (test_proba >= thr_ks).astype(int)
254 |
255 | print("\n=== TEST RESULTS (using validation KS threshold) ===")
256 | print(f"Test ROC-AUC: {test_auc:.4f}")
257 | print(f"Test PR-AUC : {test_prauc:.4f}")
258 | print(f"Test KS      : {ks_t:.4f}")
259 | print("\nTest Confusion Matrix:")
260 | print(confusion_matrix(y_test, test_pred))
261 | print("\nTest Classification Report:")
262 | print(classification_report(y_test, test_pred, digits=4))
263 |
264 |
```

Project code.py – exported 2025-08-16 01:43

```
265 | # ===== XGBoost =====
266 | X_train = X_train.astype(np.float32, copy=False)
267 | X_val   = X_val.astype(np.float32, copy=False)
268 | X_test  = X_test.astype(np.float32, copy=False)
269 |
270 | # CV setup
271 | cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=SEED)
272 | pos, neg = int((y_train == 1).sum()), int((y_train == 0).sum())
273 | scale_pos_weight = neg / max(pos, 1)
274 |
275 | pipe_xgb = Pipeline([
276 |     ("imp", SimpleImputer(strategy="median")),
277 |     ("xgb", XGBClassifier(
278 |         objective="binary:logistic",
279 |         eval_metric="auc",
280 |         tree_method="hist",
281 |         learning_rate=0.05,
282 |         n_estimators=600,
283 |         max_depth=6,
284 |         subsample=0.8,
285 |         colsample_bytree=0.8,
286 |         min_child_weight=1.0,
287 |         reg_lambda=1.0,
288 |         reg_alpha=0.0,
289 |         scale_pos_weight=scale_pos_weight,
290 |         n_jobs=THREADS,
291 |         random_state=SEED,
292 |         verbosity=0
293 |     ))
294 | ])
295 |
296 | # compact grid
297 | param_grid_xgb = {
298 |     "xgb_n_estimators": [400, 800],
299 |     "xgb_max_depth": [4, 6],
300 |     "xgb_learning_rate": [0.05, 0.1],
301 |     "xgb_min_child_weight": [1.0, 3.0],
302 | }
303 |
304 | grid_xgb = GridSearchCV(
305 |     estimator=pipe_xgb,
306 |     param_grid=param_grid_xgb,
307 |     scoring="roc_auc",
308 |     cv=cv,
```

Project code.py – exported 2025-08-16 01:43

```
309 |     n_jobs=1,
310 |     verbose=1,
311 |     refit=True
312 | )
313 |
314 | # Fit CV
315 | grid_xgb.fit(X_train, y_train)
316 | best_xgb = grid_xgb.best_estimator_
317 | print("Best params:", grid_xgb.best_params_)
318 | print("CV ROC-AUC :", grid_xgb.best_score_)
319 | gc.collect()
320 |
321 | # Validate on VAL – get KS threshold
322 | val_proba = best_xgb.predict_proba(X_val)[: , 1]
323 | val_auc    = roc_auc_score(y_val, val_proba)
324 | val_prauc = average_precision_score(y_val, val_proba)
325 | fpr, tpr, thr = roc_curve(y_val, val_proba)
326 | ks_vals = tpr - fpr
327 | ks = float(np.max(ks_vals))
328 | thr_ks = float(thr[np.argmax(ks_vals)])
329 |
330 | print(f"Validation ROC-AUC: {val_auc:.4f}")
331 | print(f"Validation PR-AUC : {val_prauc:.4f}")
332 | print(f"Validation KS      : {ks:.4f} @ threshold {thr_ks:.4f}")
333 |
334 | val_pred = (val_proba >= thr_ks).astype(int)
335 | print("\nValidation Confusion Matrix:")
336 | print(confusion_matrix(y_val, val_pred))
337 | print("\nValidation Classification Report:")
338 | print(classification_report(y_val, val_pred, digits=4))
339 | del val_proba, fpr, tpr, thr, ks_vals
340 | gc.collect()
341 |
342 | # Refit on TRAIN+VAL with best hyperparams
343 | best_xgb_final = grid_xgb.best_estimator_
344 | X_trv = pd.concat([X_train, X_val], copy=False)
345 | y_trv = pd.concat([y_train, y_val], copy=False)
346 | best_xgb_final.fit(X_trv, y_trv)
347 | del X_trv, y_trv
348 | gc.collect()
349 |
350 | # Final TEST evaluation (use validation KS threshold)
351 | test_proba = best_xgb_final.predict_proba(X_test)[: , 1]
352 | test_auc    = roc_auc_score(y_test, test_proba)
```

Project code.py – exported 2025-08-16 01:43

```
353 | test_prauc = average_precision_score(y_test, test_proba)
354 | fpr_t, tpr_t, thr_t = roc_curve(y_test, test_proba)
355 | ks_t = float(np.max(tpr_t - fpr_t))
356 | test_pred = (test_proba >= thr_ks).astype(int)
357 |
358 | print("\n=== TEST RESULTS (using validation KS threshold) ===")
359 | print(f"Test ROC-AUC: {test_auc:.4f}")
360 | print(f"Test PR-AUC : {test_prauc:.4f}")
361 | print(f"Test KS      : {ks_t:.4f}")
362 | print("\nTest Confusion Matrix:")
363 | print(confusion_matrix(y_test, test_pred))
364 | print("\nTest Classification Report:")
365 | print(classification_report(y_test, test_pred, digits=4))
366 |
367 |
368 | # ===== CatBoost =====
369 | X_train_cb = X_train.copy()
370 | X_val_cb   = X_val.copy()
371 | X_test_cb  = X_test.copy()
372 |
373 | # Class weight for imbalance
374 | pos = int((y_train == 1).sum())
375 | neg = int((y_train == 0).sum())
376 | class_weights = [1.0, neg / max(pos, 1)]
377 |
378 | # Small param grid
379 | param_grid_cb = [
380 |     {"depth": 6, "learning_rate": 0.05, "l2_leaf_reg": 3.0},
381 |     {"depth": 8, "learning_rate": 0.05, "l2_leaf_reg": 3.0},
382 |     {"depth": 6, "learning_rate": 0.1, "l2_leaf_reg": 3.0},
383 | ]
384 |
385 | best_cb = None
386 | best_auc = -np.inf
387 | best_params = None
388 |
389 | # Simple manual CV loop for small grid
390 | for params in param_grid_cb:
391 |     cb = CatBoostClassifier(
392 |         task_type="CPU",
393 |         iterations=2000,
394 |         early_stopping_rounds=100,
395 |         loss_function="Logloss",
396 |         eval_metric="AUC",
```

Project code.py – exported 2025-08-16 01:43

```
397 |         class_weights=class_weights,
398 |         random_seed=SEED,
399 |         thread_count=THREADS,
400 |         verbose=False,
401 |         **params
402 |     )
403 |     cb.fit(X_train_cb, y_train, eval_set=(X_val_cb, y_val), use_best_model=True)
404 |     val_proba = cb.predict_proba(X_val_cb)[: , 1]
405 |     val_auc = roc_auc_score(y_val, val_proba)
406 |     if val_auc > best_auc:
407 |         best_auc = val_auc
408 |         best_cb = cb
409 |         best_params = params
410 |
411 | print("Best CatBoost params:", best_params)
412 | print("Validation ROC-AUC:", best_auc)
413 |
414 | # Compute validation PR-AUC and KS
415 | val_proba = best_cb.predict_proba(X_val_cb)[: , 1]
416 | val_prauc = average_precision_score(y_val, val_proba)
417 | fpr, tpr, thr = roc_curve(y_val, val_proba)
418 | ks_vals = tpr - fpr
419 | ks = float(np.max(ks_vals))
420 | thr_ks_cb = float(thr[np.argmax(ks_vals)])
421 | print(f"Validation PR-AUC: {val_prauc:.4f}")
422 | print(f"Validation KS      : {ks:.4f} @ threshold {thr_ks_cb:.4f}")
423 |
424 | val_pred = (val_proba >= thr_ks_cb).astype(int)
425 | print("\nValidation Confusion Matrix:")
426 | print(confusion_matrix(y_val, val_pred))
427 | print("\nValidation Classification Report:")
428 | print(classification_report(y_val, val_pred, digits=4))
429 |
430 | # Retrain on TRAIN+VAL
431 | cb_final = CatBoostClassifier(
432 |     task_type="CPU",
433 |     iterations=2000,
434 |     early_stopping_rounds=100,
435 |     loss_function="Logloss",
436 |     eval_metric="AUC",
437 |     class_weights=class_weights,
438 |     random_seed=SEED,
439 |     verbose=False,
440 |     **best_params
```

Project code.py – exported 2025-08-16 01:43

```
441 | )
442 | X_trv_cb = pd.concat([X_train_cb, X_val_cb], copy=False)
443 | y_trv_cb = pd.concat([y_train, y_val], copy=False)
444 | cb_final.fit(X_trv_cb, y_trv_cb, use_best_model=False)
445 |
446 | # Test set evaluation
447 | test_proba = cb_final.predict_proba(X_test_cb)[: , 1]
448 | test_auc    = roc_auc_score(y_test, test_proba)
449 | test_prauc = average_precision_score(y_test, test_proba)
450 | fpr_t, tpr_t, thr_t = roc_curve(y_test, test_proba)
451 | ks_t = float(np.max(tpr_t - fpr_t))
452 | test_pred = (test_proba >= thr_ks_cb).astype(int)
453 |
454 | print("\n=== TEST RESULTS (CatBoost, using validation KS threshold) ===")
455 | print(f"Test ROC-AUC: {test_auc:.4f}")
456 | print(f"Test PR-AUC : {test_prauc:.4f}")
457 | print(f"Test KS      : {ks_t:.4f}")
458 | print("\nTest Confusion Matrix:")
459 | print(confusion_matrix(y_test, test_pred))
460 | print("\nTest Classification Report:")
461 | print(classification_report(y_test, test_pred, digits=4))
462 | gc.collect()
463 |
464 |
465 | # ===== Random Forest =====
466 | X_train_rf = X_train.astype(np.float32, copy=False)
467 | X_val_rf   = X_val.astype(np.float32,   copy=False)
468 | X_test_rf  = X_test.astype(np.float32,  copy=False)
469 |
470 | cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=SEED)
471 |
472 | pipe_rf = Pipeline([
473 |     ("imp", SimpleImputer(strategy="median")),
474 |     ("rf", RandomForestClassifier(
475 |         n_estimators=600,
476 |         max_depth=None,
477 |         max_features="sqrt",
478 |         min_samples_split=2,
479 |         min_samples_leaf=1,
480 |         class_weight="balanced_subsample",
481 |         n_jobs=max(1, os.cpu_count() // 2),
482 |         random_state=SEED
483 |     ))
484 | ])
```

Project code.py – exported 2025-08-16 01:43

```
485 |  
486 | # compact grid  
487 | param_grid_rf = {  
488 |     "rf__n_estimators": [400, 800],  
489 |     "rf__max_depth": [None, 12],  
490 |     "rf__min_samples_leaf": [1, 5],  
491 |     "rf__max_features": ["sqrt", 0.5],  
492 | }  
493 |  
494 | grid_rf = GridSearchCV(  
495 |     estimator=pipe_rf,  
496 |     param_grid=param_grid_rf,  
497 |     scoring="roc_auc",  
498 |     cv=cv,  
499 |     n_jobs=1,  
500 |     verbose=1,  
501 |     refit=True  
502 | )  
503 |  
504 | # Fit CV  
505 | grid_rf.fit(X_train_rf, y_train)  
506 | best_rf = grid_rf.best_estimator_  
507 | print("Best RF params:", grid_rf.best_params_)  
508 | print("CV ROC-AUC      :", grid_rf.best_score_)  
509 |  
510 | # Validate – pick KS threshold  
511 | val_proba = best_rf.predict_proba(X_val_rf)[:, 1]  
512 | val_auc   = roc_auc_score(y_val, val_proba)  
513 | val_prauc = average_precision_score(y_val, val_proba)  
514 | fpr, tpr, thr = roc_curve(y_val, val_proba)  
515 | ks_vals = tpr - fpr  
516 | ks = float(np.max(ks_vals))  
517 | thr_ks_rf = float(thr[np.argmax(ks_vals)])  
518 |  
519 | print(f"Validation ROC-AUC: {val_auc:.4f}")  
520 | print(f"Validation PR-AUC : {val_prauc:.4f}")  
521 | print(f"Validation KS      : {ks:.4f} @ threshold {thr_ks_rf:.4f}")  
522 |  
523 | val_pred = (val_proba >= thr_ks_rf).astype(int)  
524 | print("\nValidation Confusion Matrix:")  
525 | print(confusion_matrix(y_val, val_pred))  
526 | print("\nValidation Classification Report:")  
527 | print(classification_report(y_val, val_pred, digits=4))  
528 | del val_proba, fpr, tpr, thr, ks_vals
```

Project code.py – exported 2025-08-16 01:43

```
529 | gc.collect()
530 |
531 | # Refit on TRAIN+VAL with best hyperparams
532 | best_rf_final = grid_rf.best_estimator
533 | X_trv_rf = pd.concat([X_train_rf, X_val_rf], copy=False)
534 | y_trv_rf = pd.concat([y_train, y_val], copy=False)
535 | best_rf_final.fit(X_trv_rf, y_trv_rf)
536 | del X_trv_rf, y_trv_rf
537 | gc.collect()
538 |
539 | # Final TEST evaluation (use validation KS threshold)
540 | test_proba = best_rf_final.predict_proba(X_test_rf)[: , 1]
541 | test_auc = roc_auc_score(y_test, test_proba)
542 | test_prauc = average_precision_score(y_test, test_proba)
543 | fpr_t, tpr_t, thr_t = roc_curve(y_test, test_proba)
544 | ks_t = float(np.max(tpr_t - fpr_t))
545 | test_pred = (test_proba >= thr_ks_rf).astype(int)
546 |
547 | print("\n=== TEST RESULTS (Random Forest, using validation KS threshold) ===")
548 | print(f"Test ROC-AUC: {test_auc:.4f}")
549 | print(f"Test PR-AUC : {test_prauc:.4f}")
550 | print(f"Test KS      : {ks_t:.4f}")
551 | print("\nTest Confusion Matrix:")
552 | print(confusion_matrix(y_test, test_pred))
553 | print("\nTest Classification Report:")
554 | print(classification_report(y_test, test_pred, digits=4))
555 |
556 |
557 | # ===== PLOTS: ROC / PR / KS on TEST for 4 models (Logit L1, XGB, CatBoost, RF) =====
558 | # Collect test-set probabilities
559 | probas = {
560 |     "L1-Logit": best_model_final.predict_proba(X_test)[: , 1],
561 |     "XGBoost": best_xgb_final.predict_proba(X_test)[: , 1],
562 |     "CatBoost": cb_final.predict_proba(X_test_cb)[: , 1],
563 |     "RandomForest": best_rf_final.predict_proba(X_test_rf)[: , 1],
564 | }
565 |
566 | # Summary table: AUC / PR-AUC / KS (max TPR-FPR)
567 | summary_rows = []
568 | ks_curves = {}
569 | for name, p in probas.items():
570 |     fpr, tpr, thr = roc_curve(y_test, p)
571 |     ks_vals = tpr - fpr
572 |     ks = float(np.max(ks_vals))
```

Project code.py – exported 2025-08-16 01:43

```
573 |     thr_ks = float(thr[np.argmax(ks_vals)])
574 |     auc = float(roc_auc_score(y_test, p))
575 |     pr_auc = float(average_precision_score(y_test, p))
576 |     summary_rows.append([name, auc, pr_auc, ks, thr_ks])
577 |     ks_curves[name] = (thr, ks_vals)
578 |
579 | summary = pd.DataFrame(summary_rows, columns=["Model", "ROC-AUC", "PR-AUC", "KS", "KS_threshold"]) \
580 |     .sort_values("ROC-AUC", ascending=False)
581 | print("\n=== TEST summary ===")
582 | print(summary.to_string(index=False))
583 |
584 | # ROC curve
585 | plt.figure()
586 | for name, p in probas.items():
587 |     fpr, tpr, _ = roc_curve(y_test, p)
588 |     plt.plot(fpr, tpr, label=f"{name} (AUC={roc_auc_score(y_test, p):.3f})")
589 | plt.plot([0, 1], [0, 1], linestyle="--")
590 | plt.xlabel("False Positive Rate")
591 | plt.ylabel("True Positive Rate")
592 | plt.title("ROC Curves on TEST")
593 | plt.legend()
594 | plt.tight_layout()
595 | plt.savefig("roc_test.png", dpi=200)
596 | plt.show()
597 |
598 | # PR curve
599 | plt.figure()
600 | for name, p in probas.items():
601 |     prec, rec, _ = precision_recall_curve(y_test, p)
602 |     plt.plot(rec, prec, label=f"{name} (PR-AUC={average_precision_score(y_test, p):.3f})")
603 | plt.xlabel("Recall")
604 | plt.ylabel("Precision")
605 | plt.title("Precision-Recall Curves on TEST")
606 | plt.legend()
607 | plt.tight_layout()
608 | plt.savefig("pr_test.png", dpi=200)
609 | plt.show()
610 |
611 | # KS curve
612 | plt.figure()
613 | for name, (thr, ks_vals) in ks_curves.items():
614 |     plt.plot(thr, ks_vals, label=f"{name}")
615 | plt.xlabel("Threshold")
616 | plt.ylabel("KS = TPR - FPR")
```

Project code.py – exported 2025-08-16 01:43

```
617 | plt.title("KS Curves on TEST")
618 | plt.legend()
619 | plt.tight_layout()
620 | plt.savefig("ks_test.png", dpi=200)
621 | plt.show()
622 |
623 | # ===== Portfolio VaR & ES on TEST (Monte Carlo, independent defaults) =====
624 |
625 | ALPHAS = [0.95, 0.975, 0.99]
626 | NSIMS = 8000
627 |
628 | N_test = len(y_test)
629 | EAD_VEC = np.ones(N_test, dtype=float)
630 | LGD_SCALAR = 1.0
631 | W = EAD_VEC * LGD_SCALAR
632 |
633 | def simulate_portfolio_losses(p_vec, w_vec, nsims=NSIMS, seed=SEED):
634 |     """Simulate total portfolio loss nsims times for Bernoulli defaults with probs p_vec."""
635 |     rng = np.random.default_rng(seed)
636 |     n = p_vec.shape[0]
637 |     out = np.empty(nsims, dtype=float)
638 |     for s in range(nsims):
639 |         defaults = rng.random(n) < p_vec
640 |         out[s] = float(np.dot(defaults, w_vec))
641 |     return out
642 |
643 | def var_es_from_losses(losses, alpha):
644 |     """Empirical VaR and ES from a vector of simulated losses."""
645 |     var = float(np.quantile(losses, alpha, method="higher"))
646 |     es = float(losses[losses >= var].mean())
647 |     return var, es
648 |
649 | rows = []
650 |
651 | def pct_label(a: float) -> str:
652 |     x = a * 100
653 |     return f"{x:.1f}".rstrip("0").rstrip(".")
654 |
655 | for name, p in probas.items():
656 |     p = np.asarray(p, dtype=float)
657 |     seed_model = SEED + int.from_bytes(
658 |         hashlib.sha256(f"{SEED}:{name}".encode("utf-8")).digest()[:4],
659 |         "little"
660 |     )
```

Project code.py – exported 2025-08-16 01:43

```
661 |
662 |     losses = simulate_portfolio_losses(p, W, nsims=NSIMS, seed=seed_model)
663 |     meanL = float(losses.mean())
664 |     stdL = float(losses.std(ddof=1))
665 |     row = {
666 |         "Model": name,
667 |         "MeanLoss": meanL,
668 |         "StdLoss": stdL,
669 |         "MeanLossRate": meanL / np.sum(EAD_VEC)
670 |     }
671 |     for a in ALPHAS:
672 |         v, e = var_es_from_losses(losses, a)
673 |         row[f"VaR@{pct_label(a)}"] = v
674 |         row[f"ES@{pct_label(a)}"] = e
675 |         row[f"VaRate@{pct_label(a)}"] = v / np.sum(EAD_VEC)
676 |         row[f"ESrate@{pct_label(a)}"] = e / np.sum(EAD_VEC)
677 |     rows.append(row)
678 |
679 | var_es_df = pd.DataFrame(rows).sort_values(f"VaR@{pct_label(ALPHAS[-1])}", ascending=False)
680 |
681 | print("\n=== TEST Portfolio VaR & ES (counts and rates; assumes indep. defaults, EAD=1, LGD=1) ===")
682 | cols_order = ["Model", "MeanLoss", "StdLoss",
683 |               *(f for a in ALPHAS for f in (f"VaR@{pct_label(a)}", f"ES@{pct_label(a)}")),
684 |               "MeanLossRate",
685 |               *(f for a in ALPHAS for f in (f"VaRate@{pct_label(a)}", f"ESrate@{pct_label(a)}"))]
686 | print(var_es_df[cols_order].to_string(index=False))
687 |
688 |
689 | # ===== Export terminal output to pdf =====
690 | try:
691 |     from fpdf import FPDF
692 |     pdf = FPDF()
693 |     pdf.set_auto_page_break(auto=True, margin=10)
694 |     pdf.add_page()
695 |     pdf.set_font("Courier", size=9)
696 |     with open("run_output.txt", encoding="utf-8") as f:
697 |         for line in f:
698 |             pdf.multi_cell(0, 4, line.rstrip("\n")) # wraps long lines
699 |     pdf.output("run_output.pdf")
700 |     print("\nSaved PDF: run_output.pdf")
701 | except Exception as e:
702 |     print(f"\nPDF export skipped: {e}\n(Install with: pip install fpdf2)")
```